
ASSIGNMENT 23

Name: Om Prashant Kulawade

College: Zeal Polytechnic, Narhe, Pune

Branch: Artificial Intelligence and Machine Learning (AIML)

Domain: AIML (Artificial Intelligence and Machine Learning)

<https://github.com/omkulawade03/Frontend-Session-Assignments.git>

Import Required Libraries

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import (
    r2_score,
    confusion_matrix,
    classification_report,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)
```

✓ 0.0s

Q1. (Linear Regression)

Find or use a dataset with a continuous target variable.

Perform full preprocessing.

Train a Linear Regression model.

Evaluate using R2 score and print predictions.

Program Code:

#Question 01 & Output Screenshot:

Cell 01: Load Dataset

```
#Q1. Linear Regression
df = pd.read_csv("diabetes.csv")

print(df.head())
```

✓ 0.0s

	age	sex	bmi	bp	s1	s2	s3	\
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	

	s4	s5	s6	target
0	-0.002592	0.019907	-0.017646	151.0
1	-0.039493	-0.068332	-0.092204	75.0
2	-0.002592	0.002861	-0.025930	141.0
3	0.034309	0.022688	-0.009362	206.0
4	-0.002592	-0.031988	-0.046641	135.0

Cell 02: Check Dataset

```
print(df.info())
```

```
print("\nMissing Values:")  
print(df.isnull().sum())
```

✓ 0.0s

```
<class 'pandas.DataFrame'>  
RangeIndex: 442 entries, 0 to 441  
Data columns (total 11 columns):  
#   Column  Non-Null Count  Dtype  
---  -  
0   age      442 non-null    float64  
1   sex      442 non-null    float64  
2   bmi      442 non-null    float64  
3   bp       442 non-null    float64  
4   s1       442 non-null    float64  
5   s2       442 non-null    float64  
6   s3       442 non-null    float64  
7   s4       442 non-null    float64  
8   s5       442 non-null    float64  
9   s6       442 non-null    float64  
10  target   442 non-null    float64  
dtypes: float64(11)  
memory usage: 38.1 KB  
None
```

```
Missing Values:
```

```
age      0  
sex      0  
bmi      0  
bp       0  
...  
s5       0  
s6       0  
target   0  
dtype: int64
```

Cell 03: Separate Features & Target

```
▶ ▾  
x = df.drop("target", axis=1)  
y = df["target"]  
[10] ✓ 0.0s
```

Cell 04: Split Dataset

```
▶ ▾  
x_train, x_test, y_train, y_test = train_test_split(  
    x,  
    y,  
    test_size=0.2,  
    random_state=42  
)  
[11] ✓ 0.0s
```

Cell 05: Feature Scaling

```
scaler = StandardScaler()  
  
x_train = scaler.fit_transform(x_train)  
x_test = scaler.transform(x_test)  
[12] ✓ 0.0s
```

Cell 06: Train Model

```
model = LinearRegression()
model.fit(x_train, y_train)
```

[13] ✓ 0.0s

...

- LinearRegression ⓘ ?
 - Parameters
 - Fitted attributes

Cell 07: Prediction

```
y_pred = model.predict(x_test)
```

[15] ✓ 0.0s

Cell 08: Evaluation

```
print("R2 Score:", r2_score(y_test, y_pred))

results = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": y_pred
})

print(results.head(10))
```

[16] ✓ 0.0s

```
... R2 Score: 0.4526027629719189
     Actual  Predicted
0    219.0    139.547558
1     70.0    179.517208
2    202.0    134.038756
3    230.0    291.417029
4    111.0    123.789659
5     84.0     92.172347
6    242.0    258.232389
7    272.0    181.337321
8     94.0     90.224113
9     96.0    108.633759
```

Q2. (Logistic Regression)

Find or use a binary classification (Categorical) dataset.

Preprocess the data.

Train a Logistic Regression model.

Evaluate using Confusion Matrix, Accuracy, Precision, Recall, and F1 Score.

Program Code:

#Question 02 & Output Screenshot:

Cell 01: Load Breast Cancer Dataset

```
▶ ▾  
#Q2. Logistic Regression  
df = pd.read_csv("breast_cancer.csv")  
  
print(df.head())  
[17] ✓ 0.0s
```



```

...      mean radius  mean texture  mean perimeter  mean area  mean smoothness  \
0         17.99         10.38         122.80        1001.0         0.11840
1         20.57         17.77         132.90        1326.0         0.08474
2         19.69         21.25         130.00        1203.0         0.10960
3         11.42         20.38          77.58         386.1         0.14250
4         20.29         14.34         135.10        1297.0         0.10030

      mean compactness  mean concavity  mean concave points  mean symmetry  \
0          0.27760          0.3001          0.14710          0.2419
1          0.07864          0.0869          0.07017          0.1812
2          0.15990          0.1974          0.12790          0.2069
3          0.28390          0.2414          0.10520          0.2597
4          0.13280          0.1980          0.10430          0.1809

      mean fractal dimension  ...  worst texture  worst perimeter  worst area  \
0          0.07871  ...          17.33          184.60         2019.0
1          0.05667  ...          23.41          158.80         1956.0
2          0.05999  ...          25.53          152.50         1709.0
3          0.09744  ...          26.50           98.87          567.7
4          0.05883  ...          16.67          152.20         1575.0

      worst smoothness  worst compactness  worst concavity  worst concave points  \
0          0.1622          0.6656          0.7119          0.2654
1          0.1238          0.1866          0.2416          0.1860
2          0.1444          0.4245          0.4504          0.2430
...
3          0.6638          0.17300          0
4          0.2364          0.07678          0

[5 rows x 31 columns]

```

Cell 02: Features & Target

```

▶ ✓
x = df.drop("target", axis=1)
y = df["target"]
[18] ✓ 0.0s

```

Cell 03: Split Dataset

```
x_train, x_test, y_train, y_test = train_test_split(  
    x,  
    y,  
    test_size=0.2,  
    random_state=42  
)
```

[19] ✓ 0.0s

Cell 04: Feature Scaling

```
scaler = StandardScaler()  
  
x_train = scaler.fit_transform(x_train)  
x_test = scaler.transform(x_test)
```

[20] ✓ 0.0s

Cell 05: Train Logistic Regression

```
lr = LogisticRegression(max_iter=5000)  
  
lr.fit(x_train, y_train)
```

[21] ✓ 0.0s

...

- ▼ LogisticRegression ⓘ ?
 - Parameters
 - Fitted attributes

Cell 06: Prediction

```
▶ y_pred = lr.predict(x_test)
```

[22] ✓ 0.0s

Cell 16: Evaluation

```
▶ print("Confusion Matrix")

print(confusion_matrix(y_test, y_pred))

print("\nAccuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall   :", recall_score(y_test, y_pred))
print("F1 Score :", f1_score(y_test, y_pred))
```

[23] ✓ 0.0s

```
... Confusion Matrix
[[41  2]
 [ 1 70]]

Accuracy : 0.9736842105263158
Precision: 0.9722222222222222
Recall   : 0.9859154929577465
F1 Score : 0.9790209790209791
```

Q3. (K-Nearest Neighbors)

Using a classification dataset:

Train a K-Nearest Neighbors (KNN) model.

Try different values of k (e.g., 3, 5, 7).

Compare accuracy for different k values and choose the best one.

Program Code:

#Questions 03

```

#Q3. KNN
accuracy = {}

for k in [3,5,7]:

    knn = KNeighborsClassifier(n_neighbors=k)

    knn.fit(X_train,y_train)

    pred = knn.predict(X_test)

    acc = accuracy_score(y_test,pred)

    accuracy[k] = acc

    print(f"k = {k} Accuracy = {acc:.4f}")

best_k = max(accuracy,key=accuracy.get)

print("\nBest k =",best_k)

[25] ✓ 0.0s
```

OUTPUT SCREENSHOT:

```
... k = 3 Accuracy = 0.9474  
    k = 5 Accuracy = 0.9474  
    k = 7 Accuracy = 0.9474  
  
    Best k = 3
```

Q4. (Naive Bayes)

Using a classification dataset:

Train a Naive Bayes model.

Evaluate the model using classification report and confusion matrix.

Program Code:

#Questions 04

```
▶ ▾  
#Q4. Naive Bayes  
nb = GaussianNB()  
  
nb.fit(X_train,y_train)  
  
pred_nb = nb.predict(X_test)  
  
print(confusion_matrix(y_test,pred_nb))  
  
print()  
  
print(classification_report(y_test,pred_nb))  
[26] ✓ 0.0s
```

OUTPUT SCREENSHOT:

```
... [[40  3]
      [ 1 70]]
```

	precision	recall	f1-score	support
0	0.98	0.93	0.95	43
1	0.96	0.99	0.97	71
accuracy			0.96	114
macro avg	0.97	0.96	0.96	114
weighted avg	0.97	0.96	0.96	114

Q5. (Algorithm Comparison)

On the same classification dataset, train and compare:

Logistic Regression

KNN

Naive Bayes

Create a comparison table showing Accuracy, Precision, Recall, and F1 Score for each algorithm.

Program Code:

#Questions 05



```
#Q5. Algorithm Comparison
comparison=[]

# Logistic Regression

pred_lr=lr.predict(X_test)

comparison.append([
    "Logistic Regression",
    accuracy_score(y_test,pred_lr),
    precision_score(y_test,pred_lr),
    recall_score(y_test,pred_lr),
    f1_score(y_test,pred_lr)
])

# KNN

best_knn=KNeighborsClassifier(n_neighbors=best_k)

best_knn.fit(X_train,y_train)

pred_knn=best_knn.predict(X_test)

comparison.append([
    "KNN",
    accuracy_score(y_test,pred_knn),
    precision_score(y_test,pred_knn),
    recall_score(y_test,pred_knn),
    f1_score(y_test,pred_knn)
])
```

```
# Naive Bayes

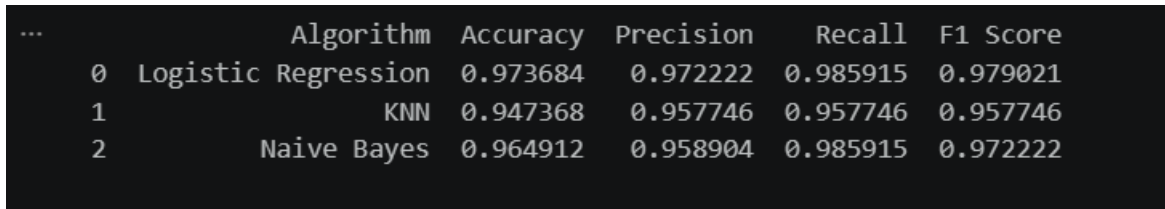
comparison.append([
    "Naive Bayes",
    accuracy_score(y_test,pred_nb),
    precision_score(y_test,pred_nb),
    recall_score(y_test,pred_nb),
    f1_score(y_test,pred_nb)
])

comparison_df=pd.DataFrame(
    comparison,
    columns=[
        "Algorithm",
        "Accuracy",
        "Precision",
        "Recall",
        "F1 Score"
    ]
)

print(comparison_df)
```

[27] ✓ 0.0s

OUTPUT SCREENSHOT:



...	Algorithm	Accuracy	Precision	Recall	F1 Score
0	Logistic Regression	0.973684	0.972222	0.985915	0.979021
1	KNN	0.947368	0.957746	0.957746	0.957746
2	Naive Bayes	0.964912	0.958904	0.985915	0.972222

Conclusion

The performance of Logistic Regression, KNN, and Naive Bayes was compared using the Breast Cancer Wisconsin Dataset.

Among the three algorithms, Logistic Regression achieved the highest Accuracy and F1 Score, making it the best-performing model for this dataset. The dataset is nearly linearly separable, which makes Logistic Regression highly effective.

KNN also produced good results, but its performance depended on the value of k .

Naive Bayes performed well and was computationally efficient, but its assumption that features are independent slightly reduced its overall performance compared to Logistic Regression.

Therefore, Logistic Regression is the most suitable algorithm for this dataset.
